

— FULL-STACK DEVELOPER • ASSESSMENT

A tournament, played *live*.

Welcome, and thanks for taking the time. Kingdom of Chess is an online chess academy for kids in India, and we're hiring full-stack developers for our learning platform. This exercise asks you to build a **tournament module with live 1-on-1 play**: coaches run tournaments, students join and play each other on a real-time board with a shared clock, and results feed a leaderboard.

We care most about a working, real-time experience and a clean path from **join a tournament** → **play a match** → **see it on the leaderboard**. This is a self-contained scenario — not production work for us.

— WHAT WE'RE LOOKING FOR

A working, real-time experience.

We want to see how you:

- Build real-time, two-way sync — move a piece here, it moves on the opponent's board.
- Keep two clients in agreement on game state.
- Auto-match waiting players into games.
- Model tournaments, matches, and results, and turn results into rankings.
- Build a clean, usable frontend with proper loading / error / empty states.
- Communicate decisions and trade-offs.

— TIME & PRIORITIES

Max. 48 hours — the earlier the better.

PRIORITISE THE LIVE MATCH FIRST

A working game — sync, turns, result — beats broad tournament-admin features. If you run low, finish the match loop and write down what you'd do next.

DO NOT REINVENT CHESS

Use `chess.js` for move legality, check/checkmate, stalemate, and PGN, and a board component (e.g. `react-chessboard`) for the UI. Your work is the sync and the data — not the rules engine.

— THE STACK • PLEASE MATCH IT

This mirrors our real environment.

BACKEND

- **NestJS 11** — TypeScript, strict mode
- **Drizzle ORM** + PostgreSQL
- **Socket.IO** for real-time
- class-validator DTOs
- Docker configuration with Compose

FRONTEND

- **Next.js** (App Router) + React 19, strict
- TailwindCSS + shadcn/ui
- TanStack React Query (REST)
- Socket.IO client (live)
- `chess.js` + a board component

Run Postgres however you like — local, Docker, Supabase/Neon free tier. Document it.

— ROLES

Two roles, access enforced on both ends.

♔ **Coach** — the organiser/admin. Creates and manages tournaments, assigns opponents (via matchmaking), and views tournament stats.

♟ **Student** — the player. Joins tournaments, plays matches live, and views the leaderboard, their ranking, and their upcoming schedule.

— REQUIRED • 01 THROUGH 09

Core requirements.

The live match is the headline; everything else clears the path to it.

01 • Auth & roles

- Email + password login issuing a **JWT in an httpOnly cookie**; protected routes.
- Role-based access: students can't hit coach-only endpoints (backend *and* frontend), and vice versa.
- A `GET /me` endpoint. The Socket.IO connection must also be **authenticated** — only a logged-in participant of a match may join that match's room.
- You may **seed** users: 1 coach and at least 4 students, with credentials in the README.

02 • Coach — tournament management

- **CRUD a tournament**: name, a time control (e.g. `5+0` = 5 min each, no increment), start date/time, and status (draft / open / ongoing / completed).
- Add students as **participants** — or let students self-join. Your call.
- **View tournament stats**: leaderboard (points), rankings, per-match results, and the winner once completed.
- **No manual pairing** — opponents are decided by auto-matchmaking.

03 • Student — join & get matched

- Browse tournaments (open / ongoing) and **join** an open one.
- Click "**Find opponent**" to enter the tournament's matchmaking queue.
- The server **auto-pairs** two waiting players into a live match and drops both straight into the game — pushed over Socket.IO, no manual refresh.
- After a game ends, a player can **re-queue** for their next opponent, and view the leaderboard, their ranking, and their match history.

MATCHMAKING RULES — THE CONCURRENCY TEST

A player is in **at most one match at a time** — never double-booked. Pairing order among eligible players can be simple FIFO/random; standings-based pairing is a stretch.

04 · Live match play — the headline feature

Once two students are auto-matched, they play in real time:

- **Two-way move sync:** a move made by one player appears on the opponent's board immediately (Socket.IO).
- **Shared clock:** each side counts down on their turn; both players see the same time. When a clock hits zero, that player loses on time.
- **PGN log:** the move list is recorded as PGN, visible during the game and persisted when it ends.
- **Game end:** handle checkmate, resignation, timeout, and stalemate/draw. The result is saved and updates the leaderboard.

MID-LEVEL BAR

A functional, working game is what we need. Making the **server authoritative** over game state and the clock — so a tampered client can't cheat or desync — is a strong signal, but it is *not* required to pass.

05 · Leaderboard & rankings

- Standard chess scoring: **win = 1, draw = ½, loss = 0**.
- A tournament leaderboard sorted by points (your tiebreak is your call — document it), with rankings and a declared winner on completion.

06 · API & real-time conventions

- Use proper API versioning.
- Request / response must be in a standardized format.
- class-validator DTOs on every REST input, with correct HTTP status codes.
- List your **Socket.IO events** (name → payload → direction) in the README.

07 · Tests

- Cover the deterministic logic: **result → leaderboard scoring, join rules, and move/turn validation**. Real-time wiring is hard to unit-test — we don't expect you to test the sockets themselves.

08 • Frontend

- Login, coach tournament management, student browse / join / schedule, the **live board + PGN panel**, and the leaderboard.
- **TanStack Query** for REST (with cache invalidation after mutations); **Socket.IO client** for live play.
- Loading, error, and empty states everywhere — nothing hangs or fails silently.

09 • README

- Setup: env, Postgres, migrations, seed, how to start both apps. Seed credentials.
- **How to play a match locally** (e.g. log in as two students in two browsers).
- The **Socket event list**.
- **Decisions & trade-offs** — explain what you did, why you did it that way, and the caveats and considerations you weighed.

— OPTIONAL • ONLY AFTER THE CORE WORKS

Stretch goals.

These separate strong submissions; they don't rescue a broken core.

- **Server-authoritative** game state — the server validates every move and is the source of truth.
- **Reconnection**: refresh or drop mid-game and rejoin with board + clock restored.
- **Spectator mode** — a coach watches a live match.
- Draw offers, threefold repetition, 50-move rule, and increment time controls (5+3).
- **Standings-based matchmaking** — pair the closest-ranked waiting players instead of FIFO.
- **Full round-robin completion** — everyone plays everyone once, with a clear complete state and declared winner.
- Move-list **navigation** — click a move to review the position.
- **DD/MM/YYYY** dates (our platform-wide format) and correct timezone handling.

— GROUND RULES

How we'd like you to work.

- **AI tools are allowed** — we use them too — but you must understand and defend every line; we'll do a short live walkthrough.
- Use `chess.js` for rules/PGN and a board library for the UI. Don't hand-roll either.
- Use **git** with small, meaningful commits.
- Keep secrets out of the repo; provide a `.env.example`.
- A **Docker Compose** one-command setup would be nice to have.
- **Migrations** must be there for the database.

— SUBMISSION

A **GitHub / GitLab repo** that must run on a clean machine from the instructions in the README.

After submission, we'll assess and get back to you with next steps. Good luck, and have fun with it. 🙌